

CS/SE 4348-005: Operating Systems Concepts Project 1

Instructor: Mingming Chen

Assigned on: Monday, September 14, 2026

Due date: Sunday, October 4, 2026

This is a group assignment. A group can consist of at most two members. Each group is expected to write the code independently and submit their own work. However, you can freely discuss the implementation ideas with other students in the class.

Students may use generative AI tools only in ways permitted by the course syllabus. In particular, AI tools may be used to help understand concepts, explain error messages, or obtain general debugging guidance, but may not be used to generate substantial portions of code or complete solutions. All submitted work must reflect the students' own understanding and effort.

Each team is required to complete a demonstration of its project to either the instructor or the TA; otherwise, the team will not receive credit. Both team members must be present and must independently demonstrate their understanding of the submitted work. Detailed demonstration requirements are provided in Section 4.

1 Project Description

Important: The goal of this project is to write a concurrent program for a UNIX/Linux-based system using processes created via the `fork()` system call.

Given an array A consisting of n numbers (which may be negative), the prefix sum problem involves computing another array B of size n such that the i th element in B is the sum of the first $i + 1$ elements in A . Formally,

$$\forall i, 0 \leq i < n : \quad B[i] = \sum_{j=0}^i A[j].$$

You have to implement the well-known Hillis and Steele concurrent algorithm to solve the prefix-sum problem using multiple worker processes. See: https://en.wikipedia.org/wiki/Prefix_sum. The algorithm is presented below for convenience.

Algorithm 1 Hillis and Steele concurrent prefix-sum algorithm

```
1:  $x[0][i] \leftarrow A[i]$  for all  $0 \leq i < n$ 
2:  $p \leftarrow 1, \quad offset \leftarrow 1$ 
3: while  $offset < n$  do
4:   for  $i \leftarrow 0$  to  $n - 1$  in parallel do
5:     if  $i < offset$  then
6:        $x[p][i] \leftarrow x[p - 1][i]$ 
7:     else
8:        $x[p][i] \leftarrow x[p - 1][i - offset] + x[p - 1][i]$ 
9:     end if
10:  end for
11:   $p \leftarrow p + 1, \quad offset \leftarrow 2 \times offset$ 
12: end while
13:  $B[i] \leftarrow x[p - 1][i]$  for all  $0 \leq i < n$ 
```

The original algorithm assumes that all array elements in each iteration can be processed in parallel. You will have to adapt it to use a given number of worker processes and divide the work among them as evenly as possible.

Your program should accept four arguments:

1. the number of elements in the input array, denoted by n ;
2. the number of worker processes, denoted by m ;
3. the input file that contains the elements in A ; and
4. the output file that will contain the elements in B .

Program Invocation and File Format

The program must be executed using the following command format:

```
./my-sum n m input_file output_file
```

where $1 \leq m \leq n$.

The input file must contain at least n whitespace-separated signed integers. The integers may appear on one or more lines. If the input file contains more than n integers, values after the first n may be ignored. You may assume that all input values and prefix sums fit in a signed 64-bit integer.

The output file must contain exactly n whitespace-separated integers representing the prefix sums in their original array order. The program should not write labels, debugging messages, or other text to the output file.

For example, suppose the program is executed as follows:

```
./my-sum 8 5 input.txt output.txt
```

and `input.txt` contains:

```
1 2 3 4 5 6 7 8
```

Then `output.txt` should contain:

```
1 3 6 10 15 21 28 36
```

In your program, the main process should create m worker (child) processes to perform the work required by the modified Hillis and Steele algorithm. In each iteration, the indices $0, \dots, n - 1$ should be divided among the m worker processes as evenly as possible. Each index should be computed by exactly one worker during that iteration. The main process is responsible for reading the input array from the input file and writing the output array to the output file. This assignment is about concurrency using processes created with `fork()`, not threads.

Hillis and Steele's algorithm works iteratively. Each iteration generates a new array, with the final iteration generating the output array. All worker processes must finish the current iteration before any worker process can start the next iteration. To achieve this synchronization, you need to implement a barrier.

A barrier provides a single method, `arriveAndWait()`. When a process invokes `arriveAndWait()`, it is blocked until all other processes have also invoked `arriveAndWait()`.

A simple non-reusable barrier is shown below.

Algorithm 2 A non-reusable barrier

```

1:  $wall[1..m] \leftarrow \{0, 0, \dots, 0\}$  ▷ shared by all processes

2: Code for process  $p_i$ :
3:  $wall[i] \leftarrow 1$ 
4: while  $\exists j : wall[j] = 0$  do
5:     skip ▷ spin
6: end while

```

Because the barrier shown above is non-reusable, a separate barrier instance must be used for each iteration of the prefix-sum algorithm. After completing its assigned indices in an iteration, each worker must invoke the corresponding barrier before beginning the next iteration. Shared variables repeatedly examined by the busy-waiting loop should be declared `volatile`.

All processes should use shared memory created with the `shmget()` system call to share all data structures, including the input array, the output array, any intermediate arrays as needed, and any data structure used to implement a barrier.

The main process must wait for all worker processes to terminate. Before the program exits, all processes must detach from shared memory, and the main process must remove all shared-memory segments.

Required Error Handling

The program must detect and handle important errors gracefully. At a minimum, it must handle the following cases:

- an incorrect number of command-line arguments;
- invalid values of n or m , including non-integer values, $n \leq 0$, $m \leq 0$, or $m > n$;
- failure to open, read from, or write to the specified files;
- an input file containing fewer than n valid integers;
- failure of a required system call, including `fork()`, `shmget()`, `shmat()`, `shmdt()`, `shmctl()`, and `wait()` or `waitpid()`; and
- abnormal termination of a worker process.

When an error prevents the program from continuing, it must print a meaningful error message to standard error, terminate with a nonzero exit status, and release any resources that it has already acquired. In particular, the main process must terminate and wait for any worker processes that cannot continue, all attached shared-memory segments must be detached, and all created shared-memory segments must be removed. The program must not crash, hang indefinitely, or leave orphaned worker processes or shared-memory segments behind.

Important: Your program should use $O(n \log_2 n)$ space and run in

$$O\left(\frac{n \log_2 n}{m} + m \log_2 n\right)$$

time to receive full credit. You may assume that $n \geq m$.

You may write your program in C or C++. Name your executable `my-sum`. Ensure that your program runs on one of the department machines `cs1.utdallas.edu`, `cs2.utdallas.edu`, or `giant.utdallas.edu`. Any significant deviation from the description may result in a penalty.

2 Implementation Constraints

The use of locks, semaphores, atomic instructions, or any other synchronization mechanism is strictly prohibited in this project. Process synchronization must be achieved exclusively through read and write operations on shared memory. No additional hardware- or software-supported synchronization primitives may be used.

3 Bonus Points

You can earn 5% bonus points by modifying the barrier algorithm to make the barrier object reusable while preserving the original structure of the algorithm. You can earn another 10% bonus points if your reusable barrier uses only $O(1)$ space and your program uses only $O(n)$ space.

4 Demonstration Requirements

Each team will have approximately **10 minutes** to demonstrate its project. The demonstration is intended to verify both program correctness and each team member's understanding of the submitted work. A prepared presentation is not required.

During the demonstration:

1. The team must compile the submitted program using its submitted **Makefile**.
2. The team must run the program using one or more test cases selected by the instructor or TA.
3. Each team member must independently answer questions about the algorithm, workload division, process creation, shared memory, barriers, error handling, and resource cleanup.
4. The instructor or TA will determine which team member answers each question.
5. The team may be asked to predict the behavior of the program for a particular input or to make a small modification to the submitted code.
6. AI tools, online assistance, communication with other students, and other external assistance may not be used during the demonstration.

The demonstration must use the version of the code submitted through Canvas. Each team member is individually responsible for understanding the complete submitted program, not only the portion that the student claims to have written. If a team member cannot demonstrate a reasonable understanding of the submitted work, that student may be required to complete an additional individual demonstration, and the student's individual project grade may be adjusted.

Teams must arrive on time and be ready to compile their programs. Technical problems during the demonstration do not extend the scheduled demonstration time. Requests to reschedule must follow the course policy regarding officially documented absences.

5 Grading

This project is worth 10 points. The 4 implementation points are normally shared by the team. The remaining 6 points evaluate each student's individual understanding during the demonstration. Therefore, members of the same team may receive different project grades.

Category	Points
Program correctness and required interface	2
Required OS design, robustness, and submission files	2
Individual demonstration questions	6
Total	10

The individual demonstration questions cover the following areas:

- the prefix-sum algorithm, iterations, offsets, and workload division;
- process creation and shared memory; and
- barrier synchronization, the submitted code, and resource cleanup.

Because AI assistance is permitted within the limits established by the course policy, the demonstration will focus primarily on each student's understanding of the submitted program. Each student must answer questions independently. The instructor or TA will determine which student answers each question, and one team member may not answer for the other.

Students may be asked to locate and explain specific portions of their code, predict the program's behavior, or describe a small modification. AI tools and other external assistance may not be used during the demonstration.

A student who does not complete the required demonstration without an approved absence will not receive credit for the project.

6 Submission Information

Submit your project through Canvas. Along with all source files, submit:

1. a **Makefile** to compile the program; and
2. a **README** file containing the names of all team members and instructions for running the compiled program.

Points will be deducted if any required files are missing. Only one team member needs to submit the files.